

Laboratorio N°2

Filtro FIR con FRDM-MCXM947

Autor: Simón Saillen

Profesores: Gustavo Parlanti, Roberto Rossi, German Molina, Pablo Ferreira

Procesamiento Digital de Señales

FCEPyN – UNC

2026

1. Introducción

2. Consigna

Realizar un programa aplicativo que sea capaz de aplicar un filtro FIR por muestras a un buffer de memoria de 512 muestras, que es adquirido con el *Laboratorio 1* teniendo en cuenta las frecuencias de muestro de 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s.

Con una de las teclas de la placa se habilitara la aplicacion del filtro FIR o se hace bypass del filtro a un buffer de salida distinto del buffer de entrada. De este buffer de salida se enviarian las muestras al DAC.

Visualizar el resultado utilizando un osciloscopio y comparar los resultados.

Requerimientos de los Filtros:

- Pasa Bajos: $f_c = 3600[\text{Hz}]$, $A_{stop} = 30[\text{dB}]$
- Pasa Altos: $f_c = 35[\text{Hz}]$, $A_{stop} = 30[\text{dB}]$
- Pasa Banda: $f_{c1} = 35[\text{Hz}]$, $f_{c2} = 3500[\text{Hz}]$, $A_{stop} = 30[\text{dB}]$
- Elimina Banda: $f_r = 50[\text{Hz}]$, $BW = 15[\text{Hz}]$, $A_{stop} = 25[\text{dB}]$

3. Desarrollo

La configuracion inicial de los pines, clocks y perifericos necesarios resulta igual a la del *Laboratorio 1*, siendo esta:

3.1. Configuración de Clocks

Principalmente se deben configurar los clocks de la placa, ya que todos los modulos de la misma dependen de esta configuracion. Para ello se utilizara dentro de ConfigTools el modulo **CLOCK** del SDK de NXP, el cual permite configurar los distintos clocks de la placa.

Clock Component	Frequency	Clock Source	Description
MAIN_clock	150 MHz	PLL0	Clock principal de la placa
ADC0_clock	48 MHz	Fast IRC	Clock del modulo ADC
DAC0_clock	48 MHz	Fast IRC	Clock del modulo DAC
CTIMER0_clock	150 MHz	PLL0	Clock del modulo Timer
FLEXXCOM4_clock	12 MHz	Slow IRC	Clock del modulo UART

Cuadro 1: Configuración de clocks usando ConfigTools

El modulo UART se configuro para enviar mensajes de debugging a la PC, por lo que se utilizo una pre configuracion con un baudrate de 115200 bps y un clock de 12 MHz.

3.2. Modulo Low-Power ADC (LPADC0)

El modulo ADC de la placa FRDM-MCXN947 es un conversor de 12 bits de baja potencia con un rango de voltaje de referencia de 0 a 3.3V. El mismo se configuro con los siguientes parametros:

Campo	Valor	Descripcion
Pre-enable ADC	Enabled	Enciende el circuito analogico con anterioridad
Autocalibration	Enabled	Calibra al iniciar para compensar desviaciones
Doze mode	Enabled	Permite que funcione en modo de bajo consumo
FIFO 0 watermark	1	Genera interrupcion cuando hay 1 muestra en la FIFO
Enable interrupt vector	Enabled	Habilita la interrupcion del ADC
Interrupt sources	FIFO 0	La fuente de interrupcion es la FIFO 0 watermark
Interrupt priority	0	Prioridad de la interrupcion del ADC

Cuadro 2: Configuración General del ADC en ConfigTools/Peripherals

Campo	Valor	Descripcion
Command ID	start_conversion	Identificador del comando
Command number	1	Numero de comando
Channel sample mode	Single end mode, using A	Modo de muestreo
Channel number	ADC0_A0	Pin de entrada del ADC
Resolution mode	High resolution (16 bits)	Resolucion de la conversion
Loop count	0	Ejecuta el comando una vez
Sample time mode	3 ADCK cycles total sample time	Tiempo de muestreo

Cuadro 3: Conversion Command del ADC en ConfigTools/Peripherals

Campo	Valor	Descripcion
Trigger ID	timer0_trigger	Identificador del trigger
Trigger number	0	Numero de trigger
Trigger command number	1	Numero de comando a ejecutar
High priority	Enabled	Ejecuta el comando con prioridad alta
Select channel A FIFO	FIFO 0	Habilita la FIFO A del ADC
Delay value	0	No hay delay entre trigger y comando

Cuadro 4: Trigger Command del ADC en ConfigTools/Peripherals

3.3. Modulo Timer (CTIMER0)

El modulo CTIMER0 es un timer de 32 bits que se utiliza para generar el trigger de conversión del ADC. Se configuró desde Peripherals en ConfigTools con los siguientes parámetros:

Campo	Valor	Descripcion
Timer mode	Timer (bus clock source)	Modo de funcionamiento
Prescaler value	1	Frecuencia de clock del timer
Match channel	0	Canal de match del timer
Match value	18750 ($\approx 8kHz$)	Valor de match del timer
Reset on match	Enabled	Resetea el timer al hacer match
Match interrupt request	Enabled	Genera interrupcion al hacer match
Interrupt priority	1	Prioridad de la interrupcion

Cuadro 5: Configuración del Timer en ConfigTools/Peripherals

El match value inicial corresponde a la velocidad de muestreo de 8 kS/s, posteriormente con el uso de los switch este valor es cambiado para cumplir con las velocidades de muestreo requeridas.

3.3.1. Segundo Modulo Timer (CTIMER1)

Tambien se configuro un segundo timer (CTIMER1) para generar la señal de parpadeo del LED RGB, el mismo se configura con un match value dependiendo del filtro seleccionado, para que el parpadeo del LED indique el filtro activo.

Su configuracion es similar a la del CTIMER0, con la diferencia de que el match value se actualiza segun el filtro seleccionado, y la prioridad de la interrupcion es menor que la del CTIMER0, para no interrumpir ni la aplicacion de filtro ni la adquisicion de muestras del ADC.

3.4. Modulo Low-Power DAC (LPDAC0)

El modulo DAC de la placa es un conversor digital a analógico de 12 bits con un rango de voltaje de salida de 0 a 3.3V. El mismo se configuró desde Peripherals en ConfigTools con los siguientes parámetros:

Campo	Valor	Descripcion
Enable DAC	Enabled	Habilita el modulo DAC
Reference voltage source	VREF_INT	Fuente de voltaje de referencia
Opamp as buffer	Enabled	Habilita el buffer de salida
Enable LPDAC	Enabled	Habilita el modo de bajo consumo
Set LPDAC level after init	Enabled	Configura el nivel de salida al encender
LPDAC level	0	Nivel de salida inicial del DAC (0V)

Cuadro 6: Configuración del DAC en ConfigTools/Peripherals

3.5. Filtros FIR (Finite Impulse Response)

La Respuesta en el Tiempo de un Filtro FIR se calcula a partir de la ponderación de las muestras presentes y pasadas con los coeficientes del filtro. Los coeficientes son la caracterizacion del filtro, variando estos se pueden obtener los distintos tipos de filtros (pasa bajos, pasa altos, etc.).

La ecuacion caracteristica de un filtro FIR es la siguiente:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k] \quad (1)$$

Teniendo:

- $y[n]$: Salida del filtro en el instante de tiempo n .
- $x[n-k]$: Entrada del filtro en el instante de tiempo $n-k$.
- $h[k]$: Coeficiente del filtro en el instante de tiempo k .
- N : Cantidad de taps (longitud del filtro).

3.6. Metodo de Ventana: Kaiser

El metodo de la ventana es el metodo mas comun para el diseño de filtros FIR debido a su simplicidad y facilidad de implementacion. Una ventana consiste en un arreglo de valores finitos que conforman los coeficientes que satisfacen los requerimientos de diseño del filtro.

La *Ventana de Kaiser* es una aproximacion a la ventana esferoidal prolata, para la que se maximiza la relacion de energia del lobulo principal y se minimiza la relación de energia del lobulo lateral.

Para lograr una atenuacion relativa del lobulo lateral de A_{stop} , el parametro β de la ventana de Kaiser se calcula con la siguiente ecuacion:

$$\beta = \begin{cases} 0,1102(A_{stop} - 8,7), & A_{stop} > 50 \\ 0,5842(A_{stop} - 21)^{0,4} + 0,07886(A_{stop} - 21), & 50 \geq A_{stop} \geq 21 \\ 0, & A_{stop} < 21 \end{cases} \quad (2)$$

Y podemos calcular el orden (o longitud) del filtro con la siguiente ecuacion:

$$N = \frac{A_{stop} - 8}{2,285 \cdot 2\pi(BW/f_s)} + 1 \quad (3)$$

El calculo de los coeficientes del filtro se realizo usando GNU/Octave, utilizando la funcion `fir1` con la ventana de Kaiser, el script utilizado se encuentra en `fir_coefficients.m`.

En la siguiente tabla se muestran los parametros de diseño de cada filtro FIR:

Filtro	Beta	Orden (N)
Pasa Bajos (LP)	2.1166	76
Pasa Altos (HP)	2.1166	151
Pasa Banda (BP)	2.1166	150
Rechaza Banda (BS)	0	571

Estos coeficientes son los que se cargan en la memoria de la placa para ser utilizados por el filtro FIR.

Los coeficientes son convertidos a formato Q15 para ser utilizados en la placa, esto se hace multiplicando los valores por 2^{15} y redondeando al entero mas cercano, luego se limita el valor a el rango de -32768 a 32767.

Los filtros diseñados se muestran en las siguientes figura, donde se puede observar la respuesta en frecuencia de cada uno de ellos en Magnitud y Fase.

Los filtros pasabanda y rechazabanda tienen similitud con los filtros pasabajos y pasaaltos debido a que la frecuencia de corte de estos filtros es muy baja en comparacion con la frecuencia de muestreo, haciendo muy poco visible la banda de paso y la banda de rechazo.

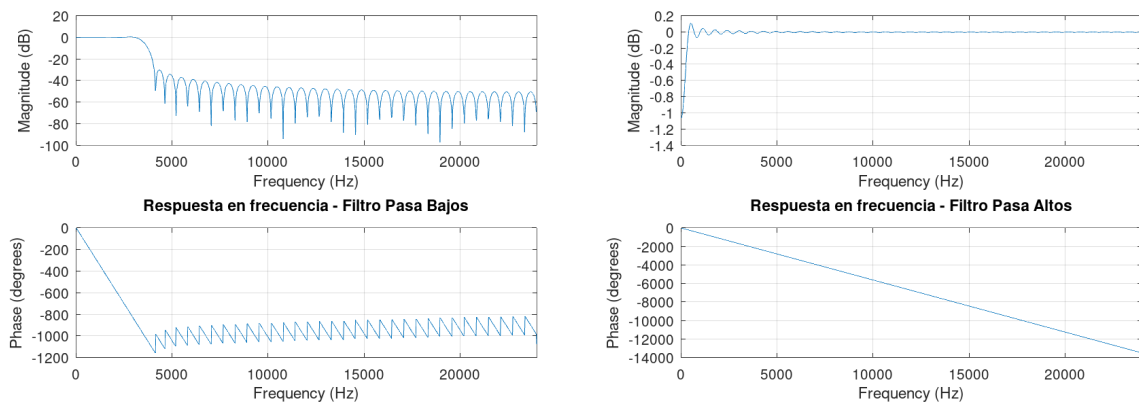


Figura 1: Filtros Pasabajo (Izquierda) y Pasaaltos (Derecha)

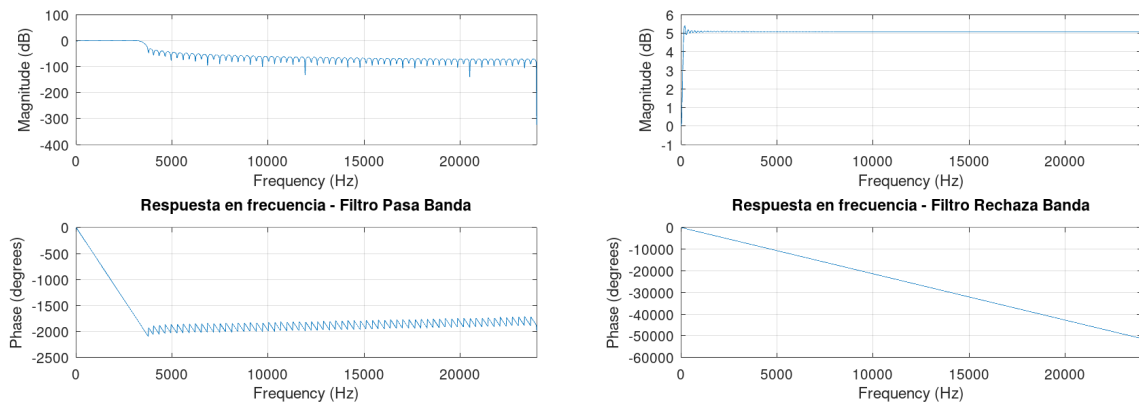


Figura 2: Filtros Pasa Banda (Izquierda) y Rechaza Banda (Derecha)

3.7. Programa Principal

El programa principal implementa un sistema de filtrado FIR en tiempo real utilizando interrupciones y buffers circulares. La arquitectura se basa en tres componentes principales:

3.7.1. Buffers de memoria

Se utilizan dos buffers circulares de 512 muestras en formato Q15:

- **input_buffer**: Almacena las muestras crudas capturadas por el ADC.
- **output_buffer**: Almacena las muestras filtradas que se envían al DAC.

3.7.2. Flujo de datos

El flujo de datos en el sistema es el siguiente:

1. El Timer0 genera un trigger periódico según la frecuencia de muestreo seleccionada.
2. La interrupción del Timer0 lee la siguiente muestra de **input_buffer**, aplica el filtro FIR seleccionado (o bypass si está deshabilitado), guarda el resultado en **output_buffer** y lo envía al DAC.
3. Al finalizar, Timer0 dispara al ADC para la siguiente conversión.
4. La interrupción del ADC almacena la muestra cruda en **input_buffer**.

El ADC siempre escribe sobre el buffer de entrada en una posición arriba de donde lee el Timer0, de esta manera se asegura que el Timer0 siempre lea un dato válido.

3.7.3. Filtros FIR con CMSIS DSP

Los filtros se implementan utilizando la librería CMSIS DSP, específicamente las funciones **arm_fir_init_q15** y **arm_fir_q15**.

El filtrado se realiza muestra por muestra (**blockSize = 1**), lo que simplifica la implementación en el contexto de interrupciones pero incrementa la sobrecarga por llamada a función.

3.7.4. Control por teclado

- **SW3 (GPIO00_IRQHandler)**: Cicla entre los filtros en orden **OFF** → **LP** → **HP** → **BP** → **BS** → **OFF**. Al cambiar de filtro, se actualiza el valor de match del Timer1 para modificar la velocidad de parpadeo del LED.
- **SW2 (GPIO01_IRQHandler)**: Cicla entre las frecuencias de muestreo: **8** → **16** → **22** → **44** → **48 kS/s**. Actualiza el valor de match del Timer0 para cambiar el periodo de muestreo.

3.7.5. Indicación visual

Para visualizar la configuración del sistema, tanto en filtro aplicado como en velocidad de muestreo, se utiliza el LED RGB en conjunto con configuraciones de parpadeo.

Las siguientes tablas muestran la correspondencia entre color y velocidad de muestreo, y entre parpadeo y filtro aplicado.

Velocidad de muestreo	Color del LED	Configuracion RGB
8 kS/s	VERDE	R = 100
16 kS/s	AZUL	B = 100
22 kS/s	AMARILLO	R = 100, G = 100
44 kS/s	ROJO	G = 100
48 kS/s	MAGENTA	R = 100, B = 100

Cuadro 7: Colores del LED RGB integrados de la placa

Filtro Aplicado	Parpadeo
OFF	Sin parpadeo
LP	1 segundo
HP	0.75 segundos
BP	0.5 segundos
BS	0.25 segundos

Cuadro 8: Parpadeo del LED RGB integrados de la placa segun el filtro aplicado

4. Resultados

4.1. Capturas de osciloscopio

4.2. Graficas de Octave

5. Conclusiones

Referencias

- [1] MathWorks (MATLAB). Kaiser Window.
<https://la.mathworks.com/help/signal/ug/kaiser-window.html>
- [2] MikroElektronika. Window Functions.
<https://www.mikroe.com/ebooks/digital-filter-design/window-functions>